

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Experiments

COURSE NAME: COMPUTER VISION

COURSE CODE:ITA0510

Slot A

COURSE OUTCOME COVERED

CO3: Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BTL 4)

Assessment Tool Weightage: 40%

QUESTIONS:3

Total Marks 30

Duration :60 Minutes

Annexure A

Experiments

Code of Conduct:

I G.Dhanusha Sree-192325012 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: G.Dhanusha sree

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks(100)
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts	
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation	
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools	
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results	
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation	

1. Preprocessing Impact Analysis

Design and perform an experiment to study the impact of preprocessing techniques such as noise removal and normalization on image quality and feature detection. Explain the theoretical concepts behind preprocessing methods and justify the selection of techniques used. Implement the preprocessing process using suitable software/tools, document the workflow and outputs systematically, and analyze how preprocessing influences feature detection accuracy and reliability.

ANSWER

Introduction

Image preprocessing is the process of improving the quality of an image before further analysis. It removes unwanted noise, enhances image clarity, and prepares the image for feature detection and object recognition.

Noise Removal

Noise is unwanted information that appears in an image due to poor lighting, camera sensors, or transmission errors. Noise removal techniques improve image quality by eliminating these unwanted pixels while preserving important details.

Common noise removal techniques include:

- Gaussian Filter
- Median Filter
- Bilateral Filter

Normalization

Normalization adjusts the intensity values of an image to a standard range. It improves image contrast and ensures consistent brightness, making feature detection more accurate.

Importance of Preprocessing

- Removes unwanted noise.
- Enhances image quality.
- Improves contrast and brightness.
- Increases feature detection accuracy.
- Makes computer vision algorithms more reliable.

Impact on Feature Detection

Preprocessing improves the visibility of important image features such as corners, edges, and textures. As a result, feature detection algorithms produce more accurate and reliable results while reducing false detections.

Code

```
import cv2

import matplotlib.pyplot as plt

# Read the uploaded image

image = cv2.imread("image.avif")

if image is None:

    print("Error: Image not loaded!")

else:

    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    # Display grayscale image

    plt.imshow(gray, cmap="gray")

    plt.title("Grayscale Image")

    plt.axis("off")

    plt.show()
```

Output



2. Image Representation Study

Conduct an experiment to convert a given image into grayscale and binary representations. Explain the significance of different image representations in computer vision applications. Design the conversion procedure using appropriate tools, document the generated outputs clearly, and analyze how image representation affects edge detection and feature extraction performance.

ANSWER

Introduction

Image representation refers to the way an image is stored and processed in a computer. Different representations are used depending on the application and processing requirements.

RGB Image

An RGB image consists of three color channels:

- Red
- Green
- Blue

It preserves full color information and is mainly used in image visualization and object recognition.

Grayscale Image

A grayscale image contains only intensity values ranging from 0 to 255. It removes color information while preserving brightness.

Advantages

- Less memory usage.
- Faster image processing.
- Suitable for edge detection and feature extraction.

Binary Image

A binary image contains only two pixel values:

- Black (0)
- White (255)

Binary images are mainly used for image segmentation and object detection.

Importance of Image Representation

- Reduces computational complexity.
- Improves processing speed.
- Simplifies image analysis.
- Helps in feature extraction and object recognition.

Effect on Edge Detection and Feature Extraction

Grayscale images preserve intensity information, making edge detection more accurate. Binary images simplify object boundaries but may lose fine details, which can affect feature extraction.

Code

```
import cv2

import matplotlib.pyplot as plt

# Read the uploaded image
image = cv2.imread("image.avif")

# Convert BGR to RGB
rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Convert to Grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Convert to Binary
_, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)

# Display the images
plt.figure(figsize=(15,5))

plt.subplot(1,3,1)

plt.imshow(rgb)

plt.title("Original Image")

plt.axis("off")
```

```
plt.subplot(1,3,2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")
plt.subplot(1,3,3)
plt.imshow(binary, cmap="gray")
plt.title("Binary Image")
plt.axis("off")
plt.show()
```

output

Original Image



Grayscale Image



Binary Image



3. Edge Detection Comparison

Design an experiment to compare Sobel and Canny edge detection techniques on the same input image. Explain the underlying concepts and operational principles of both methods. Apply the techniques using suitable tools/software, document the detected edges systematically, and analyze the comparative effectiveness of each method for different image conditions and applications.

ANSWER

Introduction

Edge detection is an important image processing technique used to identify object boundaries by detecting changes in pixel intensity. It is widely used in image segmentation, object recognition, and computer vision applications.

Sobel Edge Detection

The Sobel operator calculates the gradient of image intensity in horizontal and vertical directions to detect edges.

Advantages

- Simple and easy to implement.
- Fast processing.
- Detects strong edges.

Disadvantages

- Sensitive to noise.
- Produces thicker edges.
- Less accurate for complex images.

Canny Edge Detection

The Canny edge detector is a multi-stage algorithm that performs noise reduction, gradient calculation, non-maximum suppression, double thresholding, and edge tracking.

Advantages

- Produces thin and accurate edges.
- Less sensitive to noise.
- Better edge continuity.

- High detection accuracy.

Disadvantages

- Slightly slower than Sobel.
- More computational steps.

Comparison

- **Sobel** is faster and suitable for simple applications but is more affected by noise.
- **Canny** provides cleaner, thinner, and more accurate edges because it removes noise before detecting edges.
- Canny is preferred for applications such as medical imaging, object detection, and autonomous vehicles, while Sobel is suitable for basic image processing tasks.

Conclusion

Image preprocessing, image representation, and edge detection are fundamental steps in computer vision. Preprocessing improves image quality, grayscale representation simplifies image analysis, and Canny edge detection generally provides more accurate and reliable results than Sobel for most real-world applications.

Code

```
import cv2

import matplotlib.pyplot as plt

# Read the uploaded image
image = cv2.imread("image.avif")

# Convert to Grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Sobel Edge Detection
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
sobel = cv2.magnitude(sobel_x, sobel_y)
```



```

# Canny Edge Detection

canny = cv2.Canny(gray, 100, 200)

# Display Images

plt.figure(figsize=(15,5))

plt.subplot(1,3,1)

plt.imshow(gray, cmap='gray')

plt.title("Original Grayscale")

plt.axis("off")

plt.subplot(1,3,2)

plt.imshow(sobel, cmap='gray')

plt.title("Sobel Edge Detection")

plt.axis("off")

plt.subplot(1,3,3)

plt.imshow(canny, cmap='gray')

plt.title("Canny Edge Detection")

plt.axis("off")

plt.show()

output

```

Original Grayscale



Sobel Edge Detection



Canny Edge Detection



